

Explicação Detalhada do Código Expo SQLite CRUD

Análise passo a passo do aplicativo React Native com SQLite Assíncrono

1. Visão Geral

Este aplicativo implementa um sistema completo de cadastro, consulta, alteração e exclusão (CRUD) de usuários utilizando a API assíncrona do expo-sqlite. Os dados gerenciados são: CPF (chave primária), Nome, Idade e E-mail.

2. Importações e Recursos

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, TextInput, TouchableOpacity, FlatList, Alert } from 'react-native';
import * as SQLite from 'expo-sqlite';
```

- **useState:** Gerencia o estado dos campos de texto e a lista de registros.
- **useEffect:** Inicializa a conexão com o banco de dados assim que a tela abre.
- **expo-sqlite:** Módulo nativo para persistência de dados local.

3. Variáveis de Estado

Estado	Função
db	Armazena a conexão com o banco de dados.
nome, idade, email, cpf	Capturam os dados inseridos nos inputs.
usuarios	Guarda o array de usuários retornado do banco.

4. Inicialização do Banco de Dados

```
useEffect(() => {
  async function initDb() {
    try {
      const database = await SQLite.openDatabaseAsync('meubanco.db');
      await database.execAsync(`
        CREATE TABLE IF NOT EXISTS usuarios (
          cpf TEXT PRIMARY KEY NOT NULL,
          nome TEXT NOT NULL,
          idade INTEGER NOT NULL,
          email TEXT NOT NULL
        );
      `);
      setDb(database);
    }
  }
  initDb();
}, []);
```

```

        carregarTodos(database);
    } catch (error) {
        console.error('Erro ao inicializar o banco:', error);
    }
}
initDb();
}, []));

```

O método `openDatabaseAsync` cria/abre o banco de dados sem travar a interface do usuário. A tabela é criada com o campo `cpf` definido como `PRIMARY KEY`.

5. Operações de Banco de Dados (CRUD)

Cadastrar (Create)

```

await db.runAsync(
  'INSERT INTO usuarios (cpf, nome, idade, email) VALUES (?, ?, ?, ?);',
  [cpf.trim(), nome.trim(), parseInt(idade), email.trim()]
);

```

Consultar por Nome (Read com LIKE)

```

const query = nome.trim() === ''
  ? 'SELECT * FROM usuarios;'
  : 'SELECT * FROM usuarios WHERE nome LIKE ?;';
const params = nome.trim() === '' ? [] : [`%${nome.trim()}%`];
const resultados = await db.getAllAsync(query, params);

```

Alterar (Update)

```

const result = await db.runAsync(
  'UPDATE usuarios SET nome = ?, idade = ?, email = ? WHERE cpf = ?;',
  [nome.trim(), parseInt(idade), email.trim(), cpf.trim()]
);

```

Excluir (Delete)

```

const result = await db.runAsync(
  'DELETE FROM usuarios WHERE cpf = ?;',
  [cpf.trim()]
);

```

6. Interface e Usabilidade

- **Inputs de Texto:** Atualizam os estados através do evento `onChangeText`.

- **Botões de Ação:** Disparam as funções do CRUD individualmente.
- **Seleção Automática:** Clicar em um item da `FlatList` preenche os campos do formulário para facilitar edições.